# Oracle Spatial Option

# Spatial Option – zastosowania

- Zastosowania
  - GIS – Geographic Information System
  - CAM – Computer-Aided Manufacturing
  - CAD – Computer-Aided Design
- A co poza tym?
  - Wszelkie systemy zawierające dane pochodzące z rozproszonych geograficznie lub przestrzennie miejsc (urzędy pracy, ZUS, marketing, reklama, spedycja, lecznictwo, …)

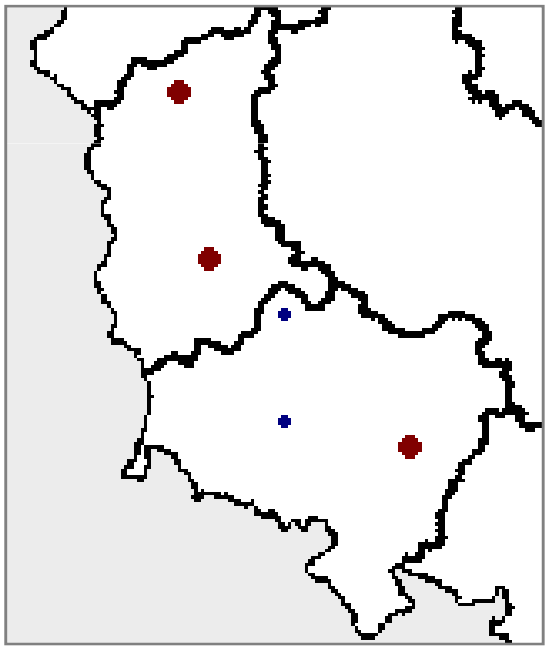
# Spatial Option – dlaczego?

- Relacje
  Dlaczego nie tak:

| Id | Województwo |
|---|---|
| 10 | dolnośląskie |
| 20 | lubuskie |
| 30 | łódzkie |

| Id_woj | Miejscowość |
|---|---|
| 20 | Zielona Góra |
| 20 | Gorzów |
| 10 | Wrocław |
| 10 | Głogów |
| 10 | Legnica |

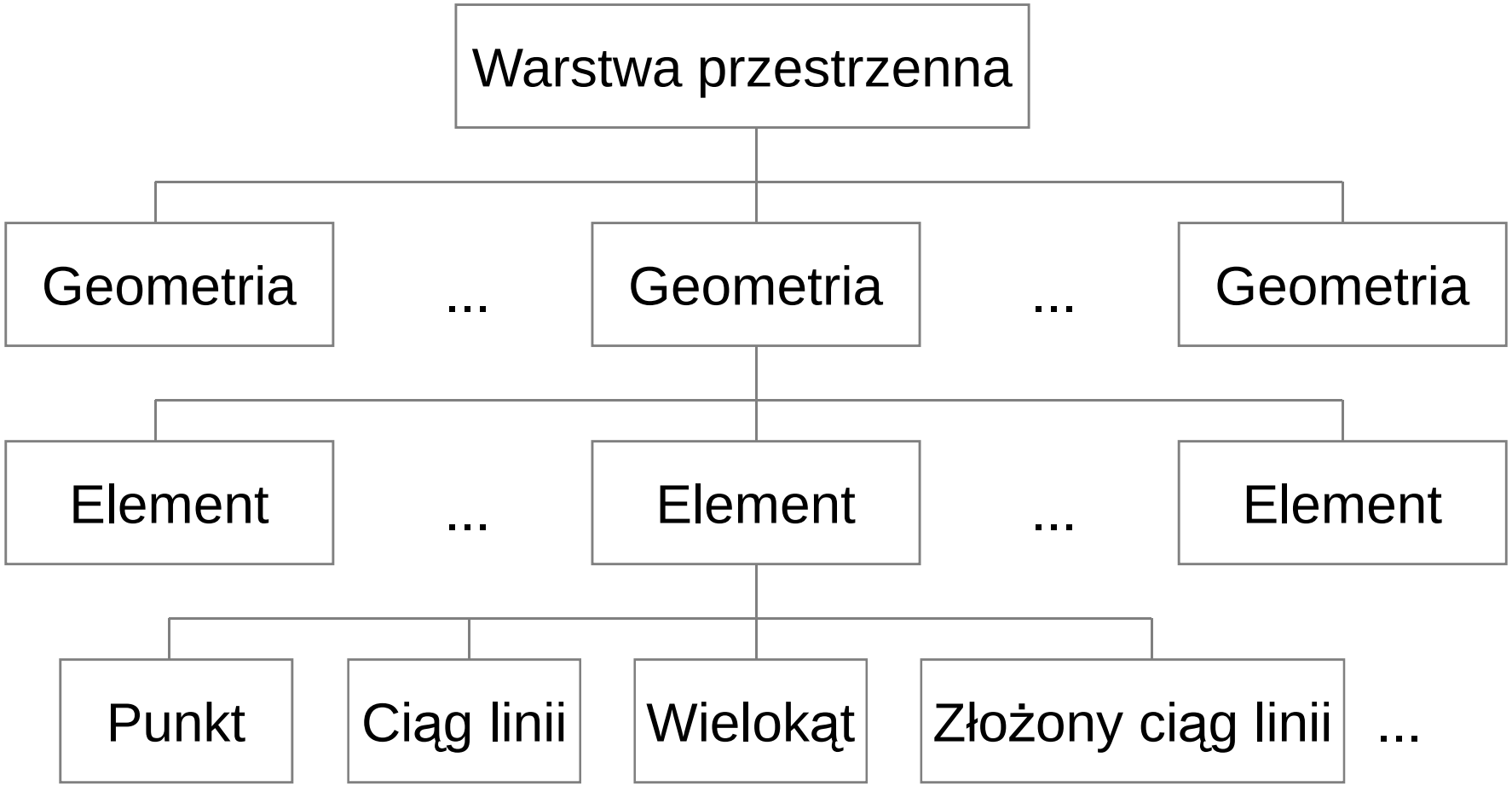
- Spatial Option
  A tak:



# Spatial Option – dlatego

| Pytanie | Relacje | Spatial Option |
|---|---|---|
| W jakim województwie leży Głogów? | ✓ | ✓ |
| Czy Gorzów jest w tym samym województwie co Zielona Góra? | ✓ | ✓ |
| Które miasto leży bliżej Zielonej Góry, Głogów czy Gorzów? | ✗ | ✓ |
| Czy województwo dolnośląskie graniczy z lubuskim? | ✗ | ✓ |
| Jakie województwa sąsiadują z województwem lubuskim | ✗ | ✓ |
| Jakie miasta znajdują się w odległości do 100 km od województwa lubuskiego? | ✗ | ✓ |
| Które województwo jest większe lubuskie czy dolnośląskie? | ✗ | ✓ |

# Spatial Option – hierarchia obiektów



# Warstwa przestrzenna (layer)

- **Jest zbiorem geometrii** posiadających podobny zestaw atrybutów (np. związanych znaczeniowo)
- Przykłady warstw przestrzennych:
  - Mapy krajów, miast;
  - Sieć dróg, sieć wodociągów, obszary leśne, obszary zurbanizowane, obszary o określonej gęstości zaludnienia;
  - Sieć sklepów, sieć hurtowni;
  - Sieć nośników reklamowych, obiekty określonego typu znaczącego dla reklamodawców;
- Z reguły warstwa jest tożsama z pojedynczą kolumną w tabeli.

# Geometria

- **Jest strukturą** przestrzenną **zbudowaną ze zbioru** prostych **elementów** geometrycznych, takich jak łuki, proste, wielokąty czy punkty.
- Jest atomowa z punktu widzenia większości operacji wykonywanych w ramach Opcji Przestrzennej
- Reprezentowana przez obiekt typu SDO_GEOMETRY
- Przykłady geometrii:
  - Wyspa, jezioro, województwo, obszar zalesiony;
  - Ulica, rzeka, granica;
  - Sklep, hurtownia, nośnik reklamowy, apteka, szkoła
  - Konstelacja gwiazd, zbiór szkół, archipelag

# Element

- **Składowa geometrii** – fragment jej opisu
- Z reguły nie występuje samodzielnie.
- Może być różnego typu:
  - Punkt,
  - Linia,
  - Wielokąt
- W zależności od typu elementu do jego opisu wykorzystywany jest jeden lub cały zbiór punktów (współrzędnych)

# Typ obiektowy - SDO_GEOMETRY

- Typ SDO_GEOMETRY – pozwalający na zdefiniowanie i przechowywanie geometrii
- Przechowywany w schemacie użytkownika MDSYS

```
MDSYS.SDO_GEOMETRY AS OBJECT (
  SDO_GTYPE     NUMBER,
  SDO_SRID      NUMBER,
  SDO_POINT     SDO_POINT_TYPE,
  SDO_ELEM_INFO MDSYS.SDO_ELEM_INFO_ARRAY,
  SDO_ORDINATES MDSYS.SDO_ORDINATE_ARRAY);
```

```
CREATE TABLE SO_WOJEWODZTWA (
  WO_ID       NUMBER(3),
  WO_NAZWA    VARCHAR2(100),
  WO_KSZTALT MDSYS.SDO_GEOMETRY);
```

# SDO_GEOMETRY

## atrybut SDO_GTYPE

```
SDO_GEOMETRY AS
  SDO_GTYPE
  SDO_SRID
  SDO_POINT
  SDO_ELEM_INFO
  SDO_ORDINATES
```

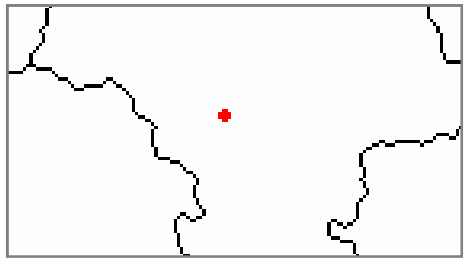
- SDO_GTYPE – **określa typ geometrii** w formacie *dltt*
  Prawidłowe wartości to:
  - *dl*00 – UNKNOWN_GEOMETRY – Wartość ignorowana.
  - *dl*01 – POINT – Geometria zawiera jeden punkt.
  - *dl*02 – LINESTRING – Geometria jeden ciąg linii.
  - *dl*03 – POLYGON – Geometria zawiera jeden wielokąt.
  - *dl*04 – COLLECTION – Geometria zawiera różnorodną kolekcję elementów.
  - *dl*05 – MULTIPOINT – Geometria składa się z wielu punktów.
  - *dl*06 – MULTILINESTRING – Geometria składa się z wielu ciągów linii.
  - *dl*07 – MULTIPOLYGON – Geometria zawiera wiele rozłącznych wielokątów

  Gdzie *d* to liczba wymiarów w których geometra występuje,
  a *l* dotyczy systemu LRS.

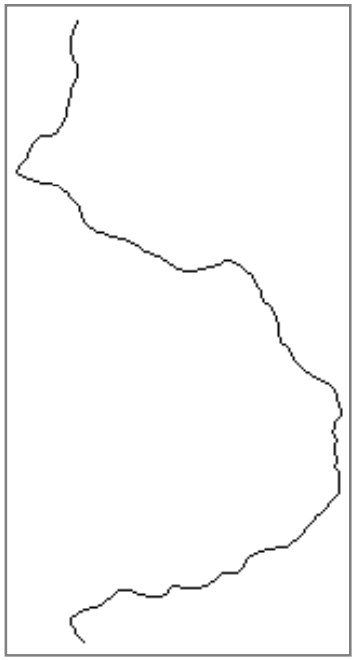
# SDO_GEOMETRY

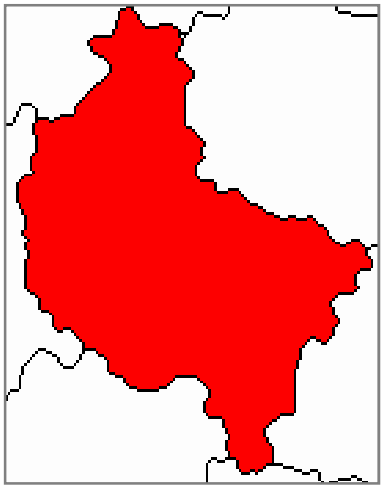
## atrybut SDO_GTYPE – przykłady

```
insert into SO_MIEJSCOWOSCI values (
  1,
  'Warszawa',
  MDSYS.SDO_GEOMETRY(
    2001, -- punkt umieszczony w
          -- układzie dwuwymiarowym
    ...);
```



```
insert into SO_RZEKI values (
  1,
  'Wisła',
  MDSYS.SDO_GEOMETRY(
    2002, -- ciąg linii
    ...);
```





```
insert into SO_WOJEWODZTWA values (
  1,
  'Wielkopolskie',
  MDSYS.SDO_GEOMETRY(
    2003, -- wielokąt
    ...);
```

# SDO_GEOMETRY

atrybut SDO_SRID

```
SDO_GEOMETRY AS
  SDO_GTYPE
  SDO_SRID
  SDO_POINT
  SDO_ELEM_INFO
  SDO_ORDINATES
```

- SDO_SRID – **definiuje układ odniesienia** (układ współrzędnych)
- Wartość pusta oznacza, że żaden układ odniesienia nie jest wykorzystywany.
  W przeciwnym przypadku SDO_SRID
  musi zawierać wartość znajdującą się w kolumnie SRID w tabeli MDSYS.CS_SRS
- Wszystkie geometrie w tej samej kolumnie muszą posiadać tą samą wartość parametru SDO_SRID

# SDO_GEOMETRY

atrybut SDO_POINT

```
SDO_GEOMETRY AS
  SDO_GTYPE
  SDO_SRID
  SDO_POINT
  SDO_ELEM_INFO
  SDO_ORDINATES
```

- SDO_POINT – zoptymalizowana **metoda przechowywania pojedynczych punktów** przestrzennych
- Jest definiowany za pomocą typu SDO_POINT_TYPE posiadającego trzy atrybuty (X,Y,Z)
- Nie jest uwzględniany gdy parametry SDO_ELEM_INFO i SDO_ORDINATES są wypełnione

# SDO_GEOMETRY

## atrybut SDO_ELEM_INFO

```
SDO_GEOMETRY AS
  SDO_GTYPE
  SDO_SRID
  SDO_POINT
  SDO_ELEM_INFO
  SDO_ORDINATES
```

- Jeżeli geometria nie jest pojedynczym punktem przechowywanym w parametrze SDO_POINT wówczas SDO_ELEM_INFO **zawiera opis** poszczególnych **elementów wchodzących w skład geometrii**.
- Każdy element opisywany jest przez trzy parametry (liczby):
  - SDO_STARTING_OFFSET – pozycja pierwszej koordynaty w SDO_ORDINATES
  - SDO_ETYPE – definiujący typ elementu
  - SDO_INTERPRETATION – interpretacja, posiada różne znaczenie w zależności od typu elementu

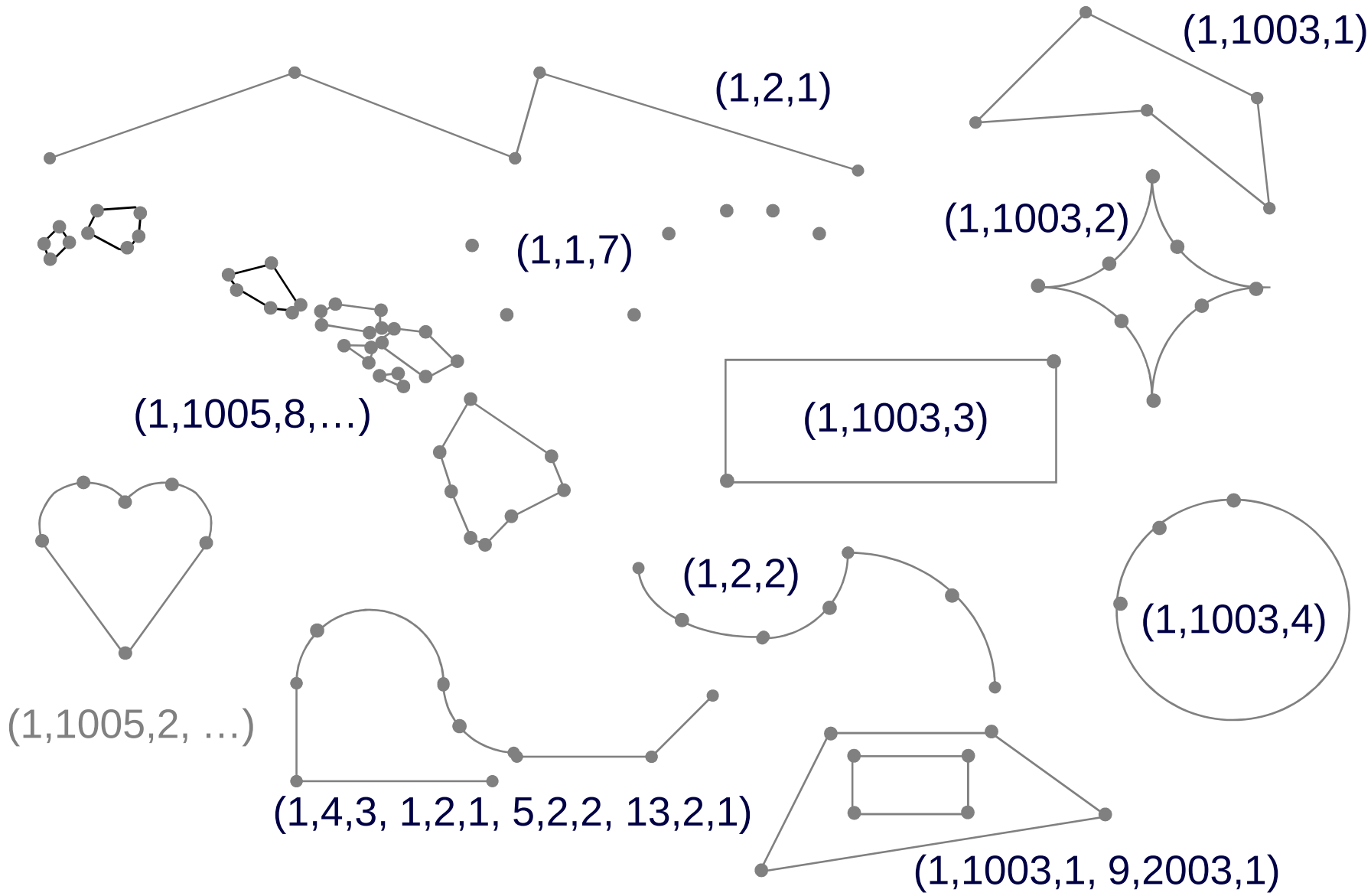
# SDO_ETYPE i SDO_INTERPRETATION

czyli typy elementów i ich interpretacje

| **Typ obiektu** | **Wartości SDO_ETYPE** | **Wartości SDO_INTERPRETATION** |
|---|---|---|
| 0 – nieznany element | 0 | |
| 1 – punkt | 1 | # – liczba punktów w kolekcji |
| 2 – ciąg linii | 2 | 1 – linie proste,<br>2 – łuki |
| 3 – wielokąt | 1003 – zewnętrzny<br>2003 – wewnętrzny | 1 – linie proste,<br>2 – łuki,<br>3 – zoptymalizowany prostokąt,<br>4 – koło |
| 4 – złożony ciąg linii | 4 | # – liczba podelementów (ciągów linii) tworzących złożony ciąg linii |
| 5 – złożony wielokąt | 1005 – zewnętrzny<br>2005 – wewnętrzny | # – liczba podelementów (ciągów linii) tworzących złożony wielokąt |

# SDO_ETYPE i SDO_INTERPRETATION

przykłady



# SDO_GEOMETRY

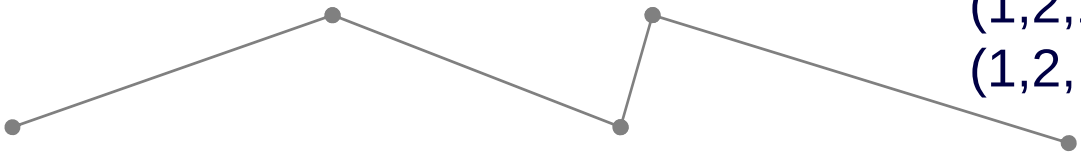
atrybut SDO_ORDINATES

```
SDO_GEOMETRY AS
  SDO_GTYPE
  SDO_SRID
  SDO_POINT
  SDO_ELEM_INFO
  SDO_ORDINATES
```

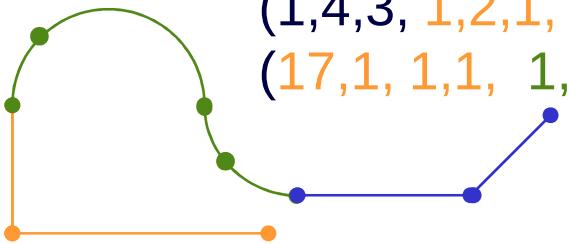
- **Zawiera koordynaty** (współrzędne poszczególnych punktów) wszystkich **elementów** tworzących geometrię
- Liczba wartości służących do opisania współrzędnych jednego punktu jest uzależniona wartości atrybutu SDO_GTYPE
- Atrybut SDO_ELEM_INFO wyznacza, które punkty w SDO_ORDINATES należą do których elementów

# SDO_GEOMETRY

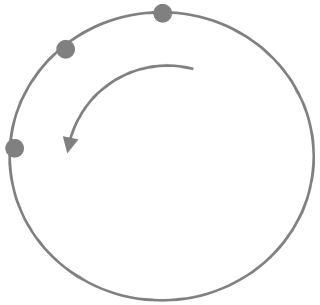
## atrybut SDO_ORDINATES – przykłady



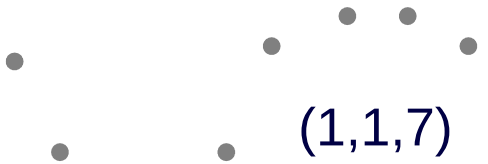
(1,2,1)
(1,2, 22,9, 41,2, 43,9, 71,1)



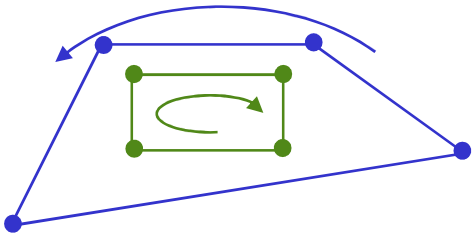
(1,4,3, 1,2,1, 5,2,2, 13,2,1)
(17,1, 1,1, 1,9, 2,14, 13,9, 15,5, 19,3, 31,3, 36, 8)



(1,1003,4) (10,20, 4,17, 1,10)



(1,1,7)
(1,12, 3,6, 14,6, 17,12, 22,15, 26,15, 30,13)



(1,1003,1, 11,2003,1)
(1,1, 30,5, 20,12, 7,12, 1,1, 8,5, 8,10, 18,10, 18,5, 8,5)

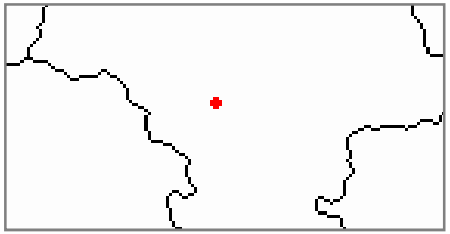
# SDO_GEOMETRY

## przykłady

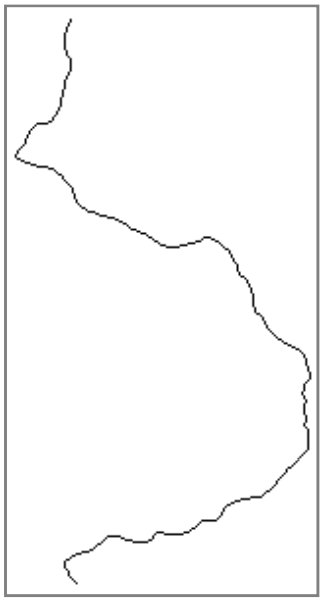
```
insert into so_miejscowosci values
(12,'Poznań',
MDSYS.SDO_GEOMETRY(2001, NULL, NULL,
MDSYS.SDO_ELEM_INFO_ARRAY(1, 1, 1),
MDSYS.SDO_ORDINATE_ARRAY(38.11, 52.50)))
```

lub

```
(12,'Poznań',
MDSYS.SDO_GEOMETRY(2001, NULL,
MDSYS.SDO_POINT_TYPE(38.11, 52.50, NULL),
NULL)
```



```
insert into so_rzeki values
(1,'Wisła',
MDSYS.SDO_GEOMETRY(2002, NULL, NULL,
MDSYS.SDO_ELEM_INFO_ARRAY(1, 2, 1),
MDSYS.SDO_ORDINATE_ARRAY(97.87, 115.15, 114.87, 123.05,
117.86, 123.61, 114.81, 113.79, 117.26, 109.79,
127.00,  97.87, 104.65,  87.04,  95.17,  96.52)))
```



```
insert into so_wojewodztwa values
(16, 'wielkopolskie',
MDSYS.SDO_GEOMETRY(2003, NULL, NULL,
MDSYS.SDO_ELEM_INFO_ARRAY(1, 1003, 1),
MDSYS.SDO_ORDINATE_ARRAY(67.06, 56.56, 65.02, 57.57, 54.12, 75.9,
54.86, 78.57, 49.45, 77.89, 47.07, 70.44, 37.56, 70.17, 31.83, 65.70,
26.08, 60.96, 24.24, 50.48, 25.40, 39.29, 29.11, 39.13, 34.61, 34.29,
32.44, 31.19, 37.59, 26.75, 43.69, 29.46, 45.38, 44.70, 54.49, 50.19,
66.72, 54.53, 66.85, 55.13, 67.06, 56.56))
```



# Metadane

- Poszczególne kolumny typu SDO_GEOMETRY mogą zostać zarejestrowane w systemie bazy danych Oracle za pomocą umieszczenia odpowiednich informacji w tzw. metadanych
- Metadane pozwalają aplikacjom podejmować decyzje odnośnie technik obsługi (np. indeksacji, wizualizacji) określonych kolumn
- Dostęp do metadanych jest możliwy za pomocą perspektyw systemowych [USER|ALL|DBA]_SDO_GEOM_METADATA
- Struktura tych perspektyw jest następująca:
  - TABLE_NAME VARCHAR2(32),
  - COLUMN_NAME VARCHAR2(32),
  - DIMINFO MDSYS.SDO_DIM_ARRAY,
  - SRID NUMBER

# Metadane

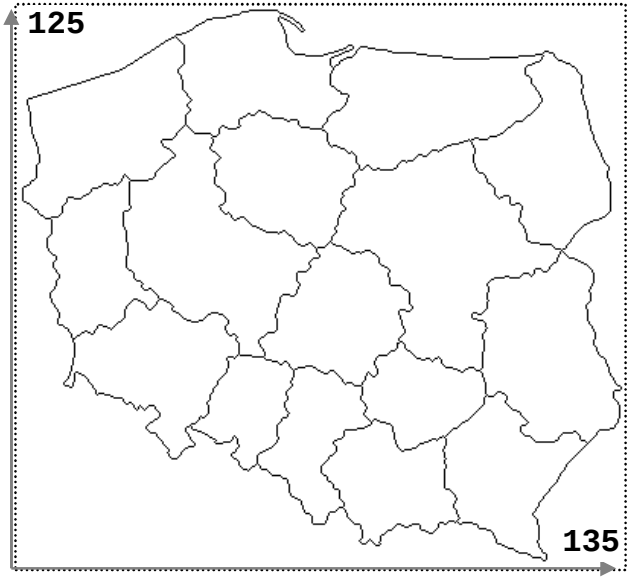
## USER_SDO_GEOM_METADATA – kolumny

- TABLE_NAME – **nazwa zarejestrowanej tabeli** np. SO_WOJEWODZTWA
- COLUMN_NAME – **nazwa kolumny** zawierającej geometrie (będącej typu SDO_GEOMETRY) np. WO_KSZTALT
- DIMINFO – (typ MDSYS.SDO_DIM_ARRAY) jest to tablica o zmiennej długości zawierająca **informacje dotyczące każdego wymiaru** występującego w geometrach. Dla każdego wymiaru przechowywane są następujące dane: nazwa wymiaru, wartość wymiaru ograniczająca geometrie z dołu, wartość ograniczająca z góry, tolerancja
- SRID – jeśli geometrie wykorzystują określony **układ odniesienia** musi on pojawić się również w tym przypadku rejestracji metadanych

# Metadane

## wskazówki i przykład

- Rejestracja metadanych jest konieczna przed utworzeniem indeksu na kolumnie zawierającej geometrie
- Rejestracja jest realizowana tylko raz dla warstwy (czyli określonej kolumny w określonej tabeli)




```
INSERT INTO USER_SDO_GEOM_METADATA
VALUES (
'SO_WOJEWODZTWA',
'WO_KSZTALT',
MDSYS.SDO_DIM_ARRAY( -- 135X125 grid
MDSYS.SDO_DIM_ELEMENT('X', 0, 135, 0.01),
MDSYS.SDO_DIM_ELEMENT('Y', 0, 125, 0.01) ),
NULL -- SRID
);
```

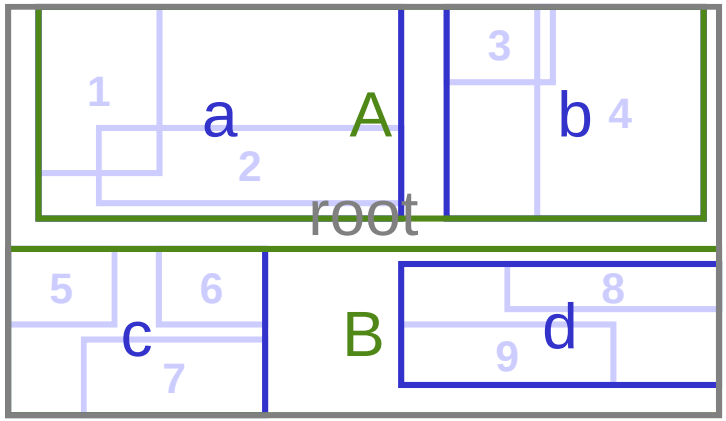
# Indeksowanie danych przestrzennych

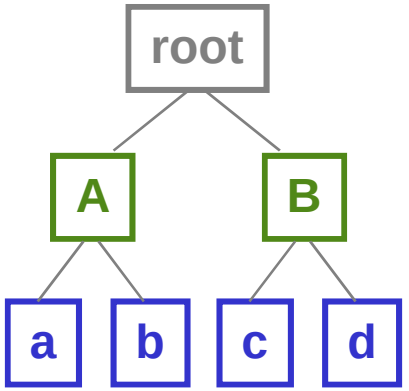
- Dane przestrzenne wymagają specjalizowanych technik indeksowania
- Oracle udostępnia dwa typy indeksów przestrzennych: R-tree i Quadtree
- Indeksy są niezbędne podczas weryfikacji zawierania się lub podczas wyznaczania zależności pomiędzy geometriami

| Własność | R-tree | Quadtree |
|---|---|---|
| Tworzenie i strojenie indeksu | Proste | Bardziej skomplikowane, dobranie odpowiednich wartości może mieć znaczące skutki |
| Wielkość indeksu | Mała | Duża |
| Zapytania z wyk. operatora najbliższego sąsiedztwa | Szybsza realizacja | Wolniejsza realizacja |
| Duża liczba modyfikacji | Zmniejszenie wydajności indeksu | Nie ma wpływu na wydajność |
| Zapytanie z wykorzystaniem SDO_WITHIN_DISTANCE | Zalecany | |
| Indeksowanych wymiarów | Do 4 | Do 2 |
| Przybliżanie geometrii | Słabe | Dobre przy odpow. właściwościach indeksu |

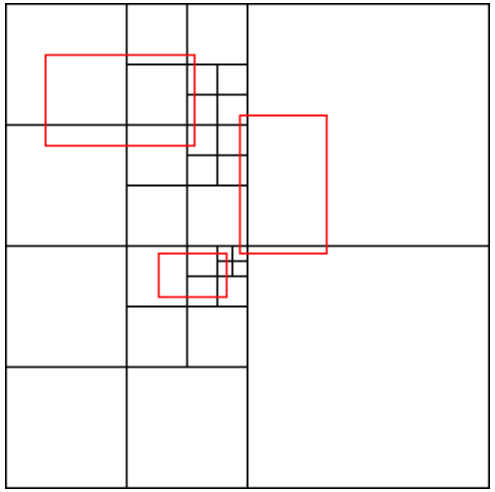
Indeksacja trzech lub więcej wymiarów powoduje, że jedynym operatorem, z którego można korzystać jest SDO_FILTER

# R-tree indeks

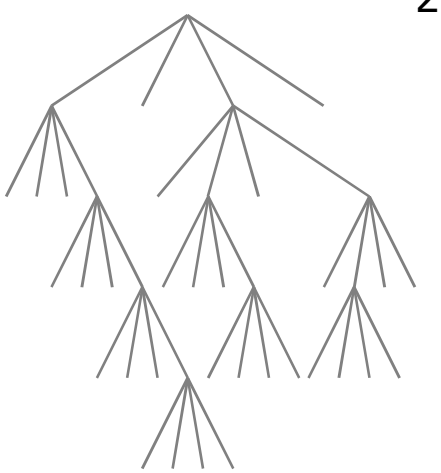



- **R-tree** indeks dokonuje przybliżenia każdej geometrii przez zastosowanie pojedynczego najmniejszego obejmującego prostokąta (MBR – minimum bounding rectangle)
- Modyfikacje tabeli mogą wpływać na degradację jakości indeksu
- Jakość indeksu można przywrócić za pomocą jego przebudowy
- Do analizy indeksu Oracle udostępnia następujące funkcje i procedury:
  - SDO_TUNE.ANALYZE_RTREE – określa czy indeks wymaga przebudowy czy nie. Wylicza obecną jego jakość i porównuje z jakością początkową
  - SDO_TUNE.RTREE_QUALITY – zwraca jakość indeksu
  - SDO_TUNE.QUALITY_DEGRADATION – zwraca stopień degradacji indeksu – stosunek powierzchni korzenia do powierzchni na poziomie 0



# Quadtree indeks



- Podstawową rolę w budowie indeksu **quadtree** odgrywa operacja zwana mozaikowaniem (tessellation)
- Każdy prostokąt dzielony jest na cztery równe podprostokąty dopóty dopóki w nie będzie dotyczył tylko jednej geometrii
- Do pokrycia wszystkich geometrii wykorzystywane są prostokąty o stałej lub zmiennej długości
- Sposób indeksowania określany jest za pomocą dwóch parametrów SDO_LEVEL i SDO_NUMTILES
  - indeksowanie stałe* – LEVEL niepuste i niezerowe, NUMTILES puste lub zerowe (stałej wielkości prostokąty) – zalecane
  - indeksowanie hybrydowe – LEVEL i NUMTILES niepuste i niezerowe (dwa zbiory prostokątów o stałej i zmiennej wielkości)
- Funkcja SDO_TUNE.ESTIMATE_TILING_LEVEL pozwala na określenie właściwego poziomu podziału

# Indeksowanie

## przykłady

```
create index WO_RTREE on SO_WOJEWODZTWA(WO_KSZTALT)
INDEXTYPE IS MDSYS.SPATIAL_INDEX; -- sdo_level=0
```

```
SQL> SELECT SDO_TUNE.RTREE_QUALITY('SCOTT','WO_RTREE') QUALITY
  2  from dual;
   QUALITY
----------
         1
SQL> SELECT SDO_TUNE.QUALITY_DEGRADATION('SCOTT','WO_RTREE') DEGRADATION
  2  from dual;
  DEGRADATION
-------------
            1
```

```
SQL> SELECT SDO_TUNE.ESTIMATE_TILING_LEVEL('SO_WOJEWODZTWA',
  2    'WO_KSZTALT',10000,'ALL_GID_EXTENT') TIL_LVL
  3  from dual;
 TIL_LVL
----------
         8
```

```
create index WO_QUADTREE on SO_WOJEWODZTWA(WO_KSZTALT)
INDEXTYPE IS MDSYS.SPATIAL_INDEX
PARAMETERS('SDO_LEVEL=8');
```

# Przykład podsumowujący (1/2)

```
CREATE TABLE geometrie1 (
id        NUMBER PRIMARY KEY,
nazwa     VARCHAR2(32),
geometria MDSYS.SDO_GEOMETRY);


INSERT INTO geometrie1 VALUES(
1,
'figura_a',
MDSYS.SDO_GEOMETRY(
2003,
NULL,
NULL,
MDSYS.SDO_ELEM_INFO_ARRAY(1,1003,3),
MDSYS.SDO_ORDINATE_ARRAY(1,1, 5,7)
)
);
```

```
INSERT INTO geometrie1 VALUES(
2,
'figura_b',
MDSYS.SDO_GEOMETRY(
2003,
NULL,
NULL,
MDSYS.SDO_ELEM_INFO_ARRAY(1,1003,1),
MDSYS.SDO_ORDINATE_ARRAY(5,1, 8,1, 8,6,
   5,7, 5,1) ) );


INSERT INTO geometrie1 VALUES(
3,
'figura_c',
MDSYS.SDO_GEOMETRY(
2003,
NULL,
NULL,
MDSYS.SDO_ELEM_INFO_ARRAY(1,1003,1),
MDSYS.SDO_ORDINATE_ARRAY(3,3, 6,3, 6,5,
   4,5, 3,3) ) );
```

# Przykład podsumowujący (2/2)

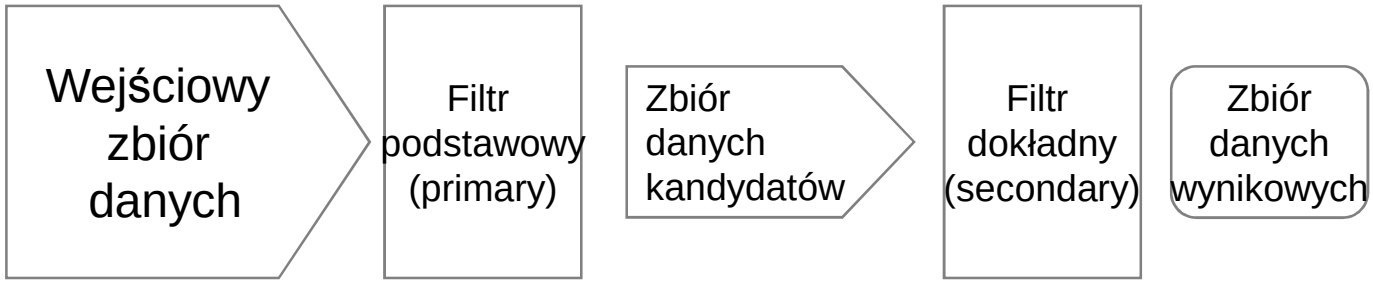
```
INSERT INTO geometrie1 VALUES(
4,
'figura_d',
MDSYS.SDO_GEOMETRY(
2003,
NULL,
NULL,
MDSYS.SDO_ELEM_INFO_ARRAY(1,100
   3,4),
MDSYS.SDO_ORDINATE_ARRAY(8,7,
   10,9, 8,11) ) );
```

```
INSERT INTO USER_SDO_GEOM_METADATA
VALUES (
'geometrie1', 'geometria',
MDSYS.SDO_DIM_ARRAY(
MDSYS.SDO_DIM_ELEMENT('X', 0, 20,
   0.005),
MDSYS.SDO_DIM_ELEMENT('Y', 0, 20,
   0.005) ),
NULL );
```

```
CREATE INDEX figura_spatial_idx
ON geometrie1(geometria)
INDEXTYPE IS MDSYS.SPATIAL_INDEX
PARAMETERS('SDO_LEVEL = 8');
```

# Zapytania

- Zapytania przestrzenne wykorzystują dwuetapowe przetwarzanie.
- Podczas pierwszego etapu z pełnego zbioru na podstawie aproksymacji wybierani są tzw. kandydaci
- W etapie drugim ze zbioru kandydatów wyznaczane są za pomocą metod dokładnych te wiersze, które spełniają zadany warunek.



# Operatory, funkcje, agregaty

- Na danych przestrzennych można operować za pomocą całego wachlarza operatorów, funkcji i agregatów.
- Podział ich jest następujący:
  - Operatory przestrzenne (ang. *Spatial Operators*)
  - Funkcje geometryczne (ang. *Geometry Functions*)
  - Przestrzenne funkcje agregujące (ang. *Spatial Aggregate Functions*)
  - Funkcje służące do transformacji układu odniesienia (ang. *Coordinate System Transformation Functions*)
  - Liniowe funkcje odniesienia (ang. *Linear Referencing Functions*)

# Operatory przestrzenne

- SDO_FILTER – sprawdza czy dwa obiekty występują w określonej zależności. Wykorzystuje tylko filtr podstawowy, wymaga indeksu.
- SDO_NN – służy do identyfikacji najbliższego sąsiedztwa, wymaga indeksu.
- SDO_NN_DISTANCE – funkcja zwracająca odległość od obiektów zwróconych przez operator SDO_NN
- SDO_RELATE – wykorzystuje indeks do wyznaczenia obiektów będących w określonym stosunku przestrzennym z geometrią
- SDO_WITHIN_DISTANCE – wykorzystuje indeks do wyznaczenia zbioru geometrii znajdującej się w ramach określonej odległości od obiektu

- Pierwszym argumentem każdego operatora jest kolumna na której założono indeks domenowy. W przypadku operatorów przestrzennych musi być to kolumna typu MDSYS.SDO_GEOMETRY, na której założono indeks MDSYS.SPATIAL_INDEX

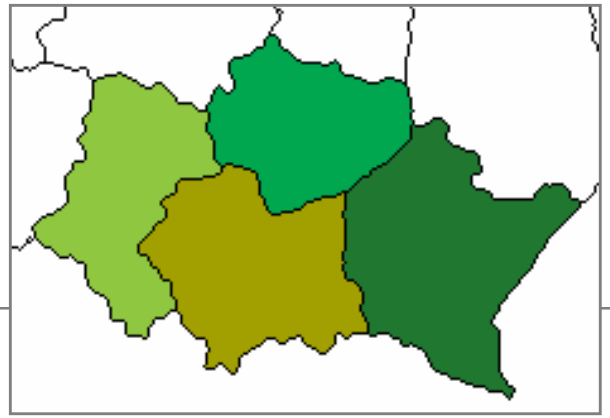
# SDO_FILTER

```
SDO_FILTER( geometry1, geometry2, params);
```

- Podstawowym i wymaganym parametrem jest **querytype**. Może on przyjmować dwie wartości: WINDOW i JOIN
  - WINDOW – zalecany w większości przypadków, może służyć do porównywania pojedynczej geometrii (2) z kolumną w tabeli (1)
  - JOIN – służy do wykonywania połączeń pomiędzy dwoma tabelami zawierającymi geometrie, rzadziej wykorzystywany

```
SQL> select a.WO_NAZWA, b.WO_NAZWA
  2  from SO_WOJEWODZTWA a, SO_WOJEWODZTWA b
  3  where SDO_FILTER(a.WO_KSZTALT, b.WO_KSZTALT,'querytype=JOIN') = 'TRUE'
  4  and b.WO_NAZWA = 'małopolskie';

WO_NAZWA        WO_NAZWA
--------------- ---------------
śląskie         małopolskie
małopolskie     małopolskie
świętokrzyskie  małopolskie
podkarpackie    małopolskie
```



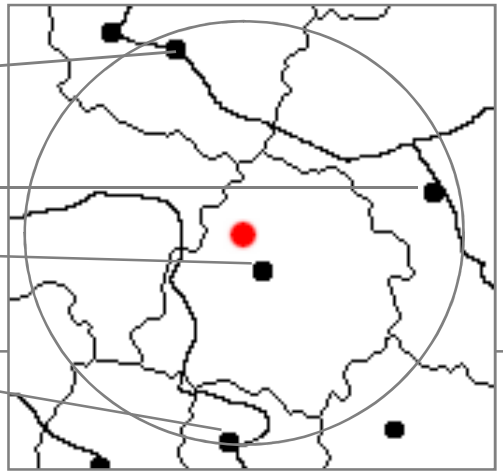
# SDO_NN

```
SDO_NN(geometry1, geometry2, param [, number]);
```

- Ciekawym parametrem operatora SDO_NN jest **sdo_num_res.** Określa on liczbę obiektów z najbliższego sąsiedztwa jaką należy zwrócić w zapytaniu

```
SQL> select MI_NAZWA
  2  from   SO_MIEJSCOWOSCI
  3  where  SDO_NN(MI_KSZTALT,MDSYS.SDO_GEOMETRY(2001, NULL, NULL,
  4                           MDSYS.SDO_ELEM_INFO_ARRAY(1, 1, 1),
  5                           MDSYS.SDO_ORDINATE_ARRAY(135/2,125/2)),
  6                'sdo_num_res=4') = 'TRUE';

MI_NAZWA
--------------------
Toruń
Warszawa
Łódź
Częstochowa
```



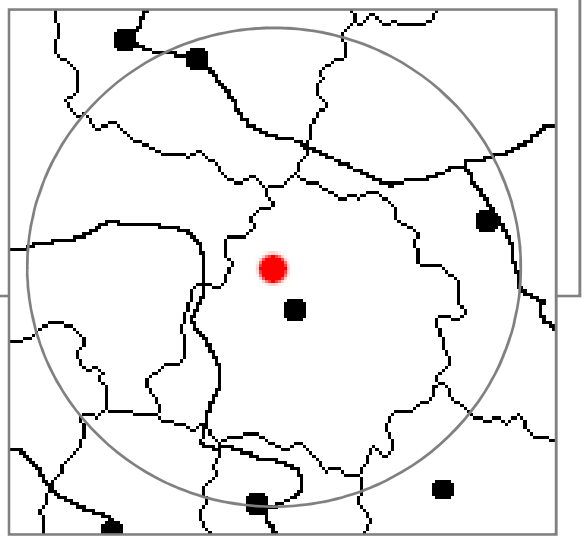
# SDO_NN_DISTANCE

```
SDO_NN_DISTANCE(number);
```

- Parametr musi być liczbą podaną w operatorze SDO_NN jako ostatni parametr

```
SQL> select MI_NAZWA, SDO_NN_DISTANCE(1) ODL
  2  from   SO_MIEJSCOWOSCI
  3  where  SDO_NN(MI_KSZTALT,MDSYS.SDO_GEOMETRY(2001, NULL, NULL,
  4                           MDSYS.SDO_ELEM_INFO_ARRAY(1, 1, 1),
  5                           MDSYS.SDO_ORDINATE_ARRAY(135/2,125/2)),
  6         'sdo_num_res=4',1) = 'TRUE';

MI_NAZWA                       ODL
-------------------- ----------
Częstochowa             25,952497
Łódź                     5,7013858
Warszawa                23,3107036
Toruń                   23,0400369
```



# SDO_RELATE

```
SDO_RELATE(geometry1, geometry2, params);
```

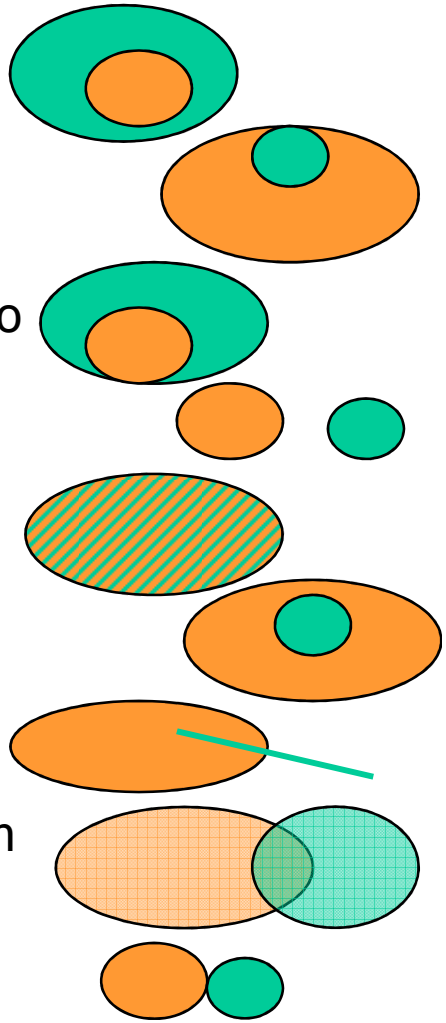
- W operatorze tym wyróżniamy dwa podstawowe parametry:
  - Mask – jedna z wielu zależności geometrycznych: TOUCH, OVERLAPBDYDISJOINT, OVERLAPBDYINTERSECT, EQUAL,INSIDE, COVEREDBY, CONTAINS, COVERS, ANYINTERACT, ON. Możliwe jest łączenie zależności za pomocą logicznego operatora OR np.: 'mask=inside OR touch'
  - Querytype – prawidłowymi wartościami są WINDOW lub JOIN (został omówiony wcześniej). Wymagany gdy *geometry2* jest kolumną tabeli

```
SQL> select a.WO_NAZWA, count(*)
  2  from   SO_WOJEWODZTWA a, SO_WOJEWODZTWA b
  3  where  SDO_RELATE(a.WO_KSZTALT,b.WO_KSZTALT,
  4         'mask=TOUCH querytype=WINDOW') = 'TRUE'
  5  group by a.WO_NAZWA;
WO_NAZWA             COUNT(*)-1
-------------------- ----------
dolnośląskie                  2
kujawsko-pomorskie            4
lubelskie                     3
lubuskie                      2
...
```

# Zależności pomiędzy geometriami

SDO_RELATE(**geometry1**, **geometry2**,...)

- ANYINTERACT – wówczas gdy obiekty nie są rozłączne.
- CONTAINS – jeśli drugi obiekt całkowicie zawiera się wewnątrz pierwszego obiektu, granice obiektów się nie stykają
- COVEREDBY – jeśli pierwszy obiekt jest całkowicie zawarty wewnątrz drugiego obiektu, granice obiektów stykają się w jednym lub wielu punktach.
- COVERS – jeśli drugi obiekt jest całkowicie wewnątrz pierwszego obiektu, granice nakładają się w jednym lub wielu miejscach.
- DISJOINT – jeśli obiekty nie mają wspólnych granic ani wewnętrznych elementów.
- EQUAL – jeśli obiekty współdzielą każdy punkt swoich granic i wnętrza włączając w to dziury wewnątrz obiektów.
- INSIDE – jeśli pierwszy obiekt całkowicie zawiera się wewnątrz drugiego obiektu, granice nie stykają się.
- OVERLAPBDYDISJOINT – jeśli obiekty zachodzą na siebie lecz nie mają części wspólnych.
- OVERLAPBDYINTERSECT – jeśli obiekty zachodzą na siebie a ich granice mają części wspólne w jednym lub wielu fragmentach.
- TOUCH – jeśli dwa obiekty współdzielą wspólne punkty graniczne, i jednocześnie nie współdzielą żadnych punktów wspólnych.



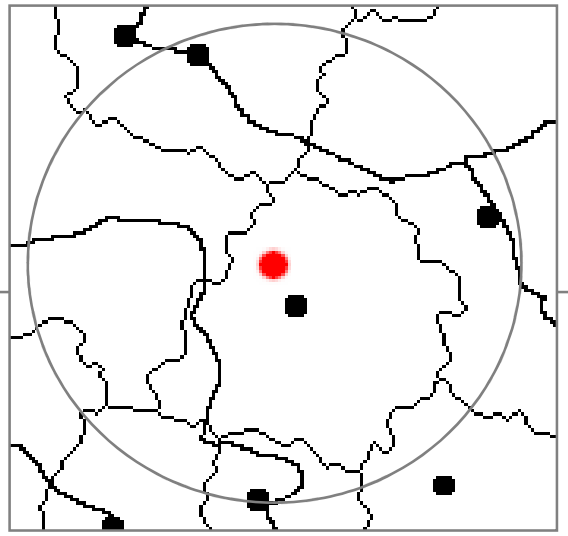
# SDO_WITHIN_DISTANCE

```
SDO_WITHIN_DISTANCE(geometry1, geometry2, params);
```

- Podstawowe parametry:
  - Distance – odległość od geometry2
  - Querytype – deklaracja dotycząca wykorzystania filtru dokładnego. Wartość równa FILTER powoduje wykorzystanie tylko filtru podstawowego

```
SQL> select MI_NAZWA
  2  from SO_MIEJSCOWOSCI
  3  where SDO_WITHIN_DISTANCE(MI_KSZTALT,MDSYS.SDO_GEOMETRY(2001, NULL, NULL,
  4                                        MDSYS.SDO_ELEM_INFO_ARRAY(1, 1, 1),
  5                                        MDSYS.SDO_ORDINATE_ARRAY(135/2,125/2)),
  6                              'distance=26') = 'TRUE';

MI_NAZWA
--------------------
Częstochowa
Łódź
Warszawa
Toruń
```



# Funkcje geometryczne

- Funkcje geometryczne można pogrupować następująco:
  - Wyznaczające relacje pomiędzy dwoma obiektami:
    - RELATE – określa zależność pomiędzy obiektami,
    - WITHIN_DISTANCE – sprawdza czy obiekty znajdują się w określonej odległości
  - Walidujące:
    - VALIDATE_GEOMETRY – sprawdza poprawność geometrii,
    - VALIDATE_LAYER – sprawdza poprawność warstwy (wszystkich geometrii w kolumnie określonej tabeli)
  - Wykonujące operacje na dwóch obiektach:
    - SDO_DISTANCE – oblicza odległość od geometrii
    - SDO_DIFFERENCE – różnica topologiczna dwóch geometrii,
    - SDO_INTERSECTION – część wspólna dwóch geometrii,
    - SDO_UNION – suma topologiczna dwóch geometrii,
    - SDO_XOR – symetryczna różnica dwóch geometrii

# Funkcje geometryczne (cd)

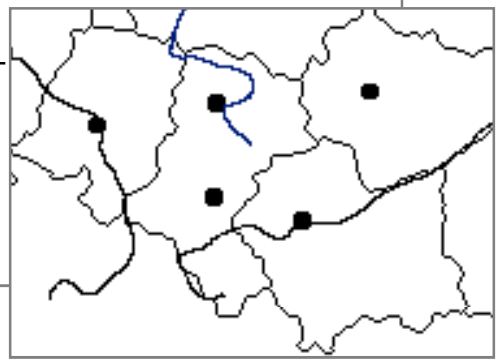
- Wykonujące operacje na pojedynczych obiektach:
  - SDO_ARC_DENSIFY – zamienia łuki i okręgi na przybliżone odpowiedniki wyrażone za pomocą linii prostych,
  - SDO_AREA – wyznacza powierzchnię dwuwymiarowej geometrii,
  - SDO_BUFFER – generuje wielokąt otaczający geometrię,
  - SDO_CENTROID – zwraca punkt georeferencyjny obrębu (tzw. controid) geometrii,
  - SDO_CONVEXHULL – zwraca obiekt reprezentujący powłokę wypukłą (convex hull) geometrii,
  - SDO_LENGTH – wylicza długość obwodu,
  - SDO_MBR – wyznacza najmniejszy prostokąt obejmujący geometrię,
  - SDO_[MIN|MAX]_MBR_ORDINATE – wyznacza "dolną" ("górną") krawędź MBR obejmującego geometrię dla określonego wymiaru,
  - SDO_POINTONSURFACE – zwraca punkt należący do powierzchni geometrii

# SDO_GEOM.RELATE

```
SDO_GEOM.RELATE(
geom1 IN MDSYS.SDO_GEOMETRY, mask  IN VARCHAR2,
geom2 IN MDSYS.SDO_GEOMETRY, tol   IN NUMBER ) RETURN VARCHAR2;
```

- Wyniki działania funkcji mogą być następujące:
  - Jeśli w parametrze mask określono oczekiwany związek pomiędzy obiektami, wówczas funkcja zwraca ciąg reprezentujący ten związek lub ciąg ‘FALSE’
  - Jeśli w parametrze mask został podany ciąg ‘DETERMINE’ wówczas funkcja daje w wyniku ciąg reprezentujący związek pomiędzy geometriami
  - W przypadku mask równego ‘ANYINTERACT’ funkcja daje w wyniku ‘TRUE’ pod warunkiem, że mamy do czynienia z nierozłącznymi geometriami

```
SQL> select a.WO_NAZWA, b.RZ_NAZWA,
  2  SDO_GEOM.RELATE(a.WO_KSZTALT, 'DETERMINE', b.RZ_KSZTALT, 0.01) RELATION
  3  from SO_WOJEWODZTWA a, SO_RZEKI b
  4  where b.RZ_NAZWA = 'Warta';
WO_NAZWA             RZ_NAZWA        RELATION
-------------------- --------------- --------------------
małopolskie          Warta           DISJOINT
podkarpackie         Warta           DISJOINT
śląskie              Warta           OVERLAPBDYDISJOINT
opolskie             Warta           OVERLAPBDYDISJOINT
...
```



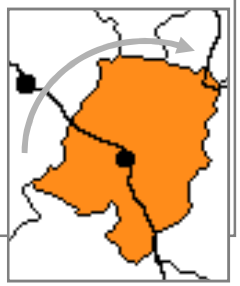
# SDO_GEOM.VALIDATE_GEOMETRY

```
SDO_GEOM.VALIDATE_GEOMETRY(
theGeometry IN MDSYS.SDO_GEOMETRY, tolerance IN NUMBER ) RETURN VARCHAR2;
```

- Funkcja weryfikująca spójność geometrii.
- Sprawdza reprezentację geometrii wg definicji elementu
- Zalecanym jej odpowiednikiem jest funkcja GEOM.VALIDATE_GEOMETRY_WITH_CONTEXT
- Wyniki działania funkcji są następujące:
  - TRUE – jeśli geometria jest prawidłowa
  - Jeśli posiada błędy wówczas uzyskujemy numer błędu Oracle'a określający przyczynę błedu
  - W przypadku funkcji …WITH_CONTEXT otrzymujemy dodatkowo kontekst błędu – informacje w jakim miejscu definicji geometrii błąd został znaleziony
- Funkcja weryfikuje zarówno spójność typu jak i spójność geometrii

```
SQL> select SDO_GEOM.VALIDATE_GEOMETRY_WITH_CONTEXT(WO_KSZTALT,0.01) VALID,
  2         WO_NAZWA from SO_WOJEWODZTWA
  3  where SDO_GEOM.VALIDATE_GEOMETRY(WO_KSZTALT,0.01) <> 'TRUE';
VALID                                          WO_NAZWA
---------------------------------------------- --------------------
13367 [Element <1>] [Ring <1>]                 opolskie
```

ORA-**13367** Wrong orientation for interior/exterior rings



# SDO_GEOM.VALIDATE_LAYER

```
SDO_GEOM.VALIDATE_LAYER_WITH_CONTEXT(
geom_table IN VARCHAR2, geom_column IN VARCHAR2,
result_table IN VARCHAR2[, commit_interval IN NUMBER]);
```

- Zalecanym odpowiednikiem jest SDO_GEOM.VALIDATE_LAYER_WITH_CONTEXT
- Procedura wypełnia tabelę wynikową informacjami rezultatach walidacji wszystkich geometrii w warstwie
- Tabela wynikowa powinna być utworzona przed wywołaniem procedury

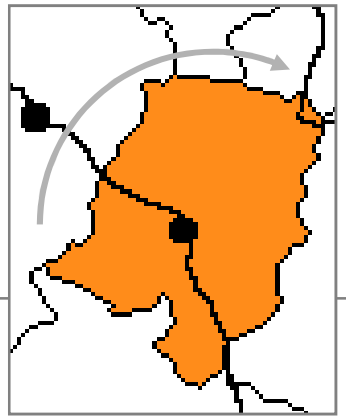
```
CREATE TABLE val_results (sdo_rowid ROWID, result varchar2(1000));

EXECUTE SDO_GEOM.VALIDATE_LAYER_WITH_CONTEXT('SO_WOJEWODZTWA','WO_KSZTALT','VAL_RESULTS');
PL/SQL procedure successfully completed.

SQL> SELECT * from val_results;

SDO_ROWID          RESULT
------------------ ----------------------------------------
                   Rows Processed <16>
AAAKl7AABAAANMyAAD 13367 [Element <1>] [Ring <1>]
```

ORA-**13367** Wrong orientation for interior/exterior rings



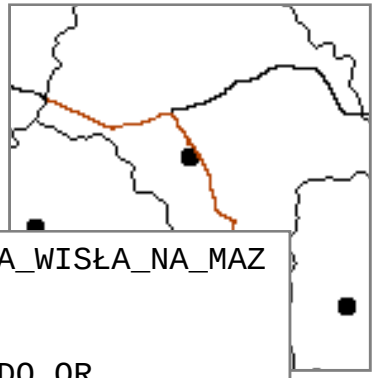
# SDO_GEOM.SDO_DISTANCE

```
SDO_GEOM.SDO_DISTANCE(
geom1 IN MDSYS.SDO_GEOMETRY,geom2 IN MDSYS.SDO_GEOMETRY,
tol IN NUMBER              [, unit IN VARCHAR2]) RETURN NUMBER;
```

```
SQL> select SDO_GEOM.SDO_DISTANCE(a.MI_KSZTALT, b.MI_KSZTALT, 0.01) ODL
  2  from SO_MIEJSCOWOSCI a, SO_MIEJSCOWOSCI b
  3  where a.MI_NAZWA = 'Zakopane'   and   b.MI_NAZWA = 'Warszawa';
       ODL
----------
63,2240603
```

# SDO_GEOM.SDO_INTERSECTION

```
SDO_GEOM.SDO_INTERSECTION(
geom1 IN MDSYS.SDO_GEOMETRY,geom2 IN MDSYS.SDO_GEOMETRY,
tol IN NUMBER ) RETURN NUMBER;
```



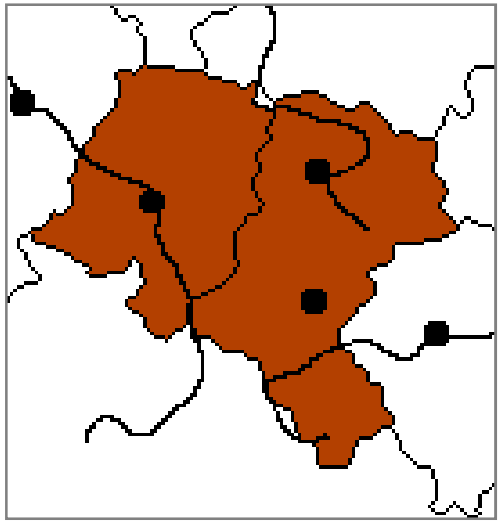
```
SQL> select SDO_GEOM.SDO_INTERSECTION(a.WO_KSZTALT, b.RZ_KSZTALT, 0.01) JAKA_WISŁA_NA_MAZ
  2  from SO_WOJEWODZTWA a, SO_RZEKI b
  3  where a.WO_NAZWA = 'mazowieckie'   and   b.RZ_NAZWA = 'Wisła';
JAKA_WISŁA_NA_MAZ(SDO_GTYPE, SDO_SRID, SDO_POINT(X, Y, Z), SDO_ELEM_INFO, SDO_OR
--------------------------------------------------------------------------------
SDO_GEOMETRY(2002, NULL, NULL, SDO_ELEM_INFO_ARRAY(1, 2, 1), SDO_ORDINATE_ARRAY(
70.8134226, 75.7885773, ..., 103.29, 48.77, 102.21, 47.45, 102.425353, 44.4465908))
```

# SDO_GEOM.SDO_UNION

```
SDO_GEOM.SDO_UNION(
geom1 IN MDSYS.SDO_GEOMETRY, geom2 IN MDSYS.SDO_GEOMETRY,
tol IN NUMBER ) RETURN MDSYS.SDO_GEOMETRY;
```

```
SQL> select SDO_GEOM.SDO_UNION(a.WO_KSZTALT, b.WO_KSZTALT, 0.01) Wielki_Śląsk
  2  from SO_WOJEWODZTWA a, SO_WOJEWODZTWA b
  3  where a.WO_NAZWA = 'opolskie' and b.WO_NAZWA = 'śląskie';

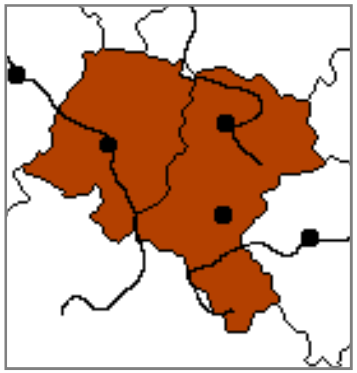
WIELKI_ŚLĄSK(SDO_GTYPE, SDO_SRID, SDO_POINT(X, Y, Z), SDO_ELEM_INFO, SDO_ORDINAT
--------------------------------------------------------------------------------
SDO_GEOMETRY(2003, NULL, NULL, SDO_ELEM_INFO_ARRAY(1, 1003, 1), SDO_ORDINATE_ARR
AY(53.51, 23.71, 60.38, 19.34, ..., 50.46, 22.35, 53.51, 23.71))
```



# SDO_GEOM.SDO_AREA

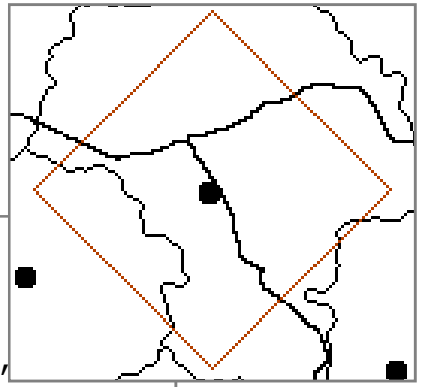
```
SDO_GEOM.SDO_AREA(
geom IN MDSYS.SDO_GEOMETRY, tol IN NUMBER[, unit IN VARCHAR2] ) RETURN NUMBER;
```

```
SQL> select SDO_GEOM.SDO_AREA(
  2           SDO_GEOM.SDO_UNION(a.WO_KSZTALT, b.WO_KSZTALT, 0.01),0.01)/
  3         SDO_GEOM.SDO_AREA(b.WO_KSZTALT, 0.01) O_ile_większy
  4  from SO_WOJEWODZTWA a, SO_WOJEWODZTWA b
  5  where a.WO_NAZWA = 'opolskie' and b.WO_NAZWA = 'śląskie';
O_ILE_WIĘKSZY
-------------
   1.75902688
```



# SDO_GEOM.SDO_BUFFER

```
SDO_GEOM.SDO_BUFFER(
geom IN MDSYS.SDO_GEOMETRY, dist IN NUMBER,
tol IN NUMBER[, params IN VARCHAR2] ) RETURN MDSYS.SDO_GEOMETRY;
```

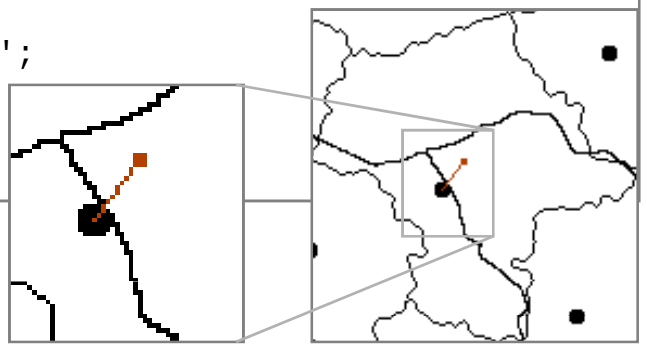


```
SQL> select SDO_GEOM.SDO_BUFFER(MI_KSZTALT,20,0.01)
  2  from   so_miejscowosci
  3  where  MI_NAZWA = 'Warszawa';
SDO_GEOM.SDO_BUFFER(MI_KSZTALT,20,0.01)(SDO_GTYPE, SDO_SRID, SDO_POINT(X,
--------------------------------------------------------------------------
SDO_GEOMETRY(2003, NULL, NULL, SDO_ELEM_INFO_ARRAY(1, 1003, 2), SDO_ORDINATE_ARR
AY(91.1, 88.07, 71.1, 68.07, 91.1, 48.07, 111.1, 68.07, 91.1, 88.07))
```

# SDO_GEOM.SDO_CENTROID

```
SDO_GEOM.SDO_CENTROID(
geom1 IN MDSYS.SDO_GEOMETRY, tol IN NUMBER) RETURN MDSYS.SDO_GEOMETRY;
```

```
SQL> select SDO_GEOM.SDO_DISTANCE(MI_KSZTALT,SDO_GEOM.SDO_CENTROID(WO_KSZTALT,0.01),0.01) ODL
  2  from   so_miejscowosci, so_wojewodztwa
  3  where  MI_NAZWA = 'Warszawa' and WO_NAZWA = 'mazowieckie';
       ODL
----------
5.20148519
```



# SDO_GEOM.SDO_LENGTH

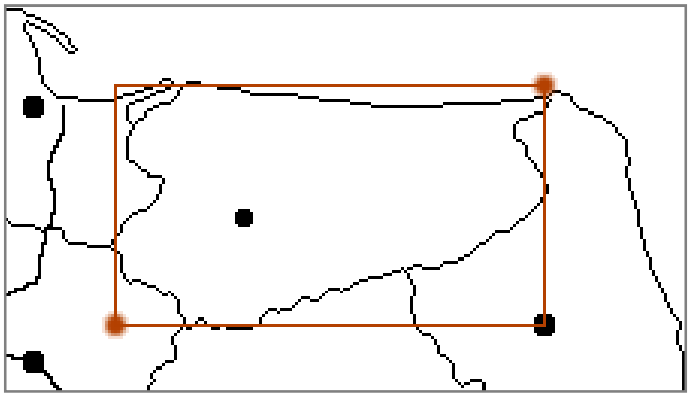
```
SDO_GEOM.SDO_LENGTH( geom1 IN MDSYS.SDO_GEOMETRY,
tol IN NUMBER, [, unit IN VARCHAR2]) RETURN NUMBER;
```

```
SQL> select SDO_GEOM.SDO_LENGTH(WO_KSZTALT,0.01) OBWOD
  2  from   so_wojewodztwa
  3  where  WO_NAZWA = 'mazowieckie';
     OBWOD
----------
169.274219
```

# SDO_GEOM.SDO_MBR

```
SDO_GEOM.SDO_MBR( geom IN MDSYS.SDO_GEOMETRY
[, dim IN MDSYS.SDO_DIM_ARRAY] ) RETURN MDSYS.SDO_GEOMETRY;
```

```
SQL> select SDO_GEOM.SDO_MBR(WO_KSZTALT) MBR
  2  from   so_wojewodztwa
  3  where  WO_NAZWA = 'warmińsko-mazurskie';
MBR(SDO_GTYPE, SDO_SRID, SDO_POINT(X, Y, Z), SDO_ELEM_INFO, SDO_ORDINATES)
--------------------------------------------------------------------------------
SDO_GEOMETRY(2003, NULL, NULL, SDO_ELEM_INFO_ARRAY(1, 1003, 3), SDO_ORDINATE_ARR
AY(67.73, 89.07, 116.84, 115.49))
```



# Przestrzenne funkcje agregujące

- SDO_AGGR_CENTROID* – zwraca punkt georeferencyjny (środek ciężkości) zbioru geometrii
- SDO_AGGR_CONVEXHULL* – obiekt reprezentujący powłokę wypukłą (convex hull) zbioru geometrii,
- SDO_AGGR_LRS_CONCAT* – wyznacza geometrię LRS będącą konkatenacją zbioru geometrii LRS
- SDO_AGGR_MBR – wyznacza najmniejszy prostokąt obejmujący zbiór geometrii
- SDO_AGGR_UNION* – wyznacza topologiczną sumę zbioru geometrii

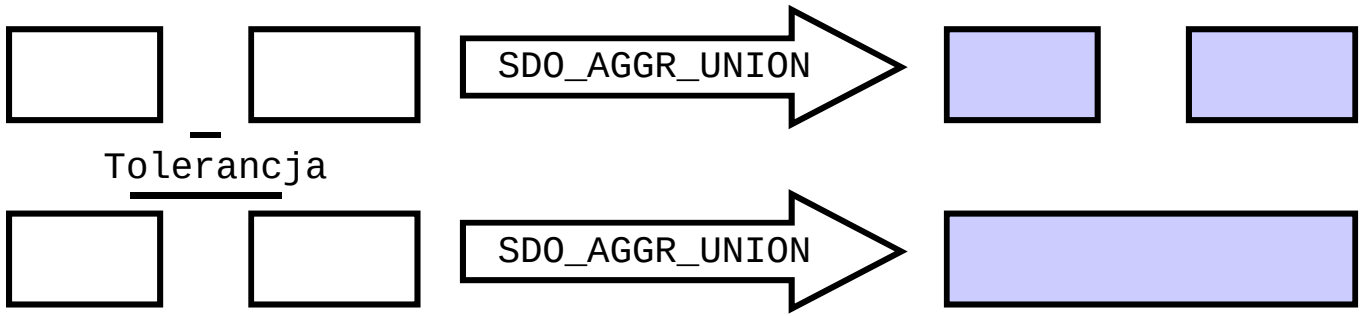
* - Większość funkcji agregujących akceptuje parametr typu MDSYS.SDOAGGRTYPE a nie bezpośrednio MDSYS.SDO_GEOMETRY.

# Typ MDSYS.SDO_GEOMETRY

- Wiele funkcji agregujących przyjmuje jako parametr obiekt typu MDSYS.SDO_GEOMETRY
- Definicja tego typu jest następująca:

```
CREATE TYPE sdoaggrtype AS OBJECT (
geometry MDSYS.SDO_GEOMETRY,
tolerance NUMBER);
```

- Tolerancja powinna być taka sama jak zarejestrowana w ramach metadanych określonej kolumny. Ma ona wpływ na wyniki działania funkcji agregujących



# SDO_AGGR_UNION

```
SDO_AGGR_UNION(
AggregateGeometry MDSYS.SDOAGGRTYPE) RETURN MDSYS.SDO_GEOMETRY;
```

```
SQL> select SDO_AGGR_UNION(MDSYS.SDOAGGRTYPE(WO_KSZTALT,0.001)) A_TO_POLSKA_WLASNIE
  2  from   so_wojewodztwa;

A_TO_POLSKA_WLASNIE(SDO_GTYPE, SDO_SRID, SDO_POINT(X, Y, Z), SDO_ELEM_INFO, SDO_
--------------------------------------------------------------------------------
SDO_GEOMETRY(2003, NULL, NULL, SDO_ELEM_INFO_ARRAY(1, 1003, 1), SDO_ORDINATE_ARR
AY(7.45, 78.23, 7.11, 71.12, 10.09, 54.39, 12.87, 51.48, 10.84, 40.98, 12.73, 42
.02, 12.87, 43.69, 18.2                                  , 32.43, 28.38, 33.8,
 25.7, 35.56, 24.72, 39                                  17, 46.08, 28.32, 49.
11, 29.13, 48.43, 26.08                                   19.34, 66.04, 10.16,
 72.47, 13.89, 75.18, 1                                  , 6.46, 80.94, 7.45,
87.68, 9.89, 89.75, 9.1                                  , 2.49, 117.86, 2.37,
 114.81, 12.19, 117.1,                                   1.72, 31.3, 134.45, 4
0.64, 133.1, 41.66, 126                                  3.14, 121.24, 71.12,
132.08, 79.93, 132.08,                                   6, 118.57, 115.25, 11
6.84, 114.81, 91.08, 11                                  69.77, 112.44, 73.19,
 115.38, 60.28, 116.5,                                    124.09, 42.4, 122.42
, 35.22, 118.87, 16.63,                                  2.64, 81.54, 7.45, 78
.23))
```

